TEHNICA PROGRAMĂRII DINAMICE

1. Plata unei sume folosind un număr minim de monede cu valori date

Considerând faptul că avem la dispoziție n monede cu valorile $v_1, v_2, \dots, v_n$ pe care putem să le folosim pentru a plăti o sumă P, trebuie să determinăm o modalitate de plată a sumei date folosind un număr minim de monede (vom presupune faptul că avem la dispoziție un număr suficient de monede din fiecare tip).

De exemplu, dacă avem la dispoziție $n = 3$ tipuri de monede cu valorile $v = (2\$, 3\$, 5\$)$, atunci putem să plătim suma $P = 12\$$ în 5 moduri: $4 \times 3\$, 1 \times 2\$ + 2 \times 5\$$, $2 \times 2\$ + 1 \times 3\$ + 1 \times 5\$, 3 \times 2\$ + 2 \times 3\$$ și $6 \times 2\$$. Evident, numărul minim de monede pe care putem să-l folosim este 3, corespunzător variantei $1 \times 2\$ + 2 \times 5\$$.

Pentru a genera toate modalitățile de plată a unei sume folosind monede cu valori date se poate utiliza tehnica Backtracking, algoritmul fiind deja prezentat în capitolul dedicat tehnicii de programare respective. Modificând algoritmul respectiv, putem determina și o modalitate de plată a unei sume folosind un număr minim de monede (pentru fiecare modalitate de plată vom calcula numărul de monede utilizate și vom reține modalitatea cu număr minim de monede), dar algoritmul va avea o complexitate exponențială, deci va fi ineficient!

Fiind o problemă de optim, putem încerca și o rezolvare de tip Greedy, respectiv să utilizăm pentru plata sumei, la fiecare pas, un număr maxim de monede cu cea mai valoare dintre cele neutilizate deja pentru plata sumei. Pentru exemplul de mai sus, vom considera monedele în ordinea descrescătoare a valorilor lor, respectiv $v = (5\$, 3\$, 2\$)$, și vom plăti suma $P = 12\$$, astfel:

- utilizăm 2 monede cu valoarea de 5$, deci suma de plată rămasă devine $P = 2\$$;
- nu putem utiliza nicio monedă cu valoarea de 3$;
- utilizăm o monedă cu valoarea de 2$, deci suma de plată rămasă devine $P = 0\$$ și algoritmul se termină cu succes;
- numărul de monede utilizate, respectiv 3 monede, este minim.

Totuși, această rezolvare de tip Greedy nu va furniza rezultatul optim în orice caz. De exemplu, dacă monedele au valorile $v = (5\$, 4\$, 1\$)$ și suma de plată este $P = 8\$$, folosind algoritmul Greedy vom găsi următoarea soluție:

- utilizăm o monedă cu valoarea de 5$, deci suma de plată rămasă devine $P = 3\$$;
- nu putem utiliza nicio monedă cu valoarea de 4$;
- utilizăm 3 monede cu valoarea de 1$, deci suma de plată rămasă devine $P = 0\$$ și algoritmul se termină cu succes;
- numărul de monede utilizate, respectiv 4 monede, nu este minim (numărul minim de monede se obține când se utilizează două monede cu valoarea de 4$).

Se observă faptul că existența monedei cu valoarea de 1$ permite algoritmului Greedy să găsească întotdeauna o soluție, chiar dacă aceasta nu este optimă. Totuși, sunt cazuri în care algoritmul Greedy nu va găsi nicio soluție, deși problema are cel puțin una. De exemplu, dacă monedele au valorile $v = (5\$, 4\$, 2\$)$ și suma de plată este tot $P = 8\$$, vom proceda astfel:

- utilizăm o monedă cu valoarea de 5$, deci suma de plată rămasă devine $P = 3\$$;
- nu putem utiliza nicio monedă cu valoarea de 4$;

- utilizăm o monedă cu valoarea de 2$, deci suma de plată rămasă devine $P = 1\$$;
- deoarece nu mai există alte tipuri de monede, algoritmul se termină fără să găsească o soluție, optimă sau nu!

Deoarece această problema are o importanță practică deosebită, în anumite țări sunt utilizate așa-numitele *sisteme canonice de valori pentru monede*, care permit algoritmului Greedy prezentat mai sus (numit și *algoritmul casierului*) să furnizeze o soluție optimă pentru orice sumă de plată (https://www.cs.princeton.edu/courses/archive/spring07/cos423/lectures/greed-dp.pdf).

Pentru a rezolva problema folosind metoda programării dinamice, observăm faptul că numărul minim de monede necesare pentru a plăti o sumă P folosind o monedă cu valoarea x (evident, $1 \leq x \leq P$) se obține adăugând 1 la numărul minim de monede necesar pentru a plăti suma $P - x$ utilizând toate tipurile de monede disponibile. De exemplu, numărul minim de monede necesare pentru a plăti suma $P = 12\$$ folosind o monedă cu valoarea $x = 5\$$ se obține adăugând 1 la numărul minim de monede necesare pentru a plăti suma $P - x = 7\$$ utilizând toate tipurile de monede disponibile. Deoarece suma $P - x$ poate să aibă orice valoare cuprinsă între 0 și $P - 1$, rezultă că pentru a putea calcula numărul minim de monede necesare pentru a plăti suma P folosind o monedă cu valoarea x trebuie să cunoaștem numărul minim de monede necesare pentru a plăti orice sumă cuprinsă între 0 și $P - 1$ folosind toate tipurile de monede disponibile cu valorile $v_1, v_2, \dots, v_n$. Generalizând această observație pentru toate tipurile de monede date, observăm faptul că numărul minim de monede necesare pentru a plăti o sumă P folosind toate tipurile de monede se obține adăugând 1 la minimul dintre: numărul minim de monede necesare pentru a plăti suma $P - v_1$, numărul minim de monede necesare pentru a plăti suma $P - v_2$,..., numărul minim de monede necesare pentru a plăti suma $P - v_n$ (evident, se vor lua în considerare doar cazurile în care moneda cu valoare v_i poate fi utilizată pentru plata sumei P, adică $v_i \leq P$).

Considerând o listă $nrmin$ cu $P + 1$ elemente de tip întreg în care elementul $nrmin[i]$ va reține numărul minim de monede necesare pentru a plăti suma i, cuprinsă între 0 și P, folosind toate tipurile de monede disponibile, relația de recurență care caracterizează substructura optimală a problemei este următoarea:

$$nrmin[i] = \begin{cases} 0, & \text{pentru } i = 0 \\ 1 + \min\limits_{0 \leq j < n} \{nrmin[i - v[j]] \mid v[j] \leq i\}, & \text{pentru } 1 \leq i \leq P \end{cases}$$

Inițial, toate elementele listei $nrmin$ trebuie să aibă valoarea "$+\infty$", adică o valoare strict mai mare decât orice valoare posibilă pentru elementele sale. Deoarece numărul maxim de monede pe care îl putem folosi pentru a plăti suma maximă P este chiar P (valoarea minimă a unei monede este 1$!), vom inițializa toate elementele listei $nrmin$ cu $P + 1$.

Soluția problemei, adică numărul minim de monede necesare pentru a plăti suma P, este dată de valoarea $nrmin[P]$, dacă ea este diferită de valoarea de inițializare $P + 1$, altfel, dacă rămâne egală cu P+1, înseamnă că suma P nu poate fi plătită folosind monede cu valorile date. Pentru a reconstitui mai ușor o modalitate optimă de plată a sumei P vom folosi o listă $pred$, tot cu $P + 1$ elemente de tip întreg, în care un element $pred[i]$ va conține valoarea -1 dacă nu există nicio modalitate de plată a sumei i folosind monedele date sau va conține valoarea monedei $v[j]$ utilizată pentru a plăti suma i cu un număr

minim de monede, adică valoarea $v[j]$ pentru care s-a obținut $\min_{0 \le j < n} \{nrmin[i - v[j]] | v[j] \le i\}$.

Considerând exemplul dat, cu $P = 12\$$ și $v = (2\$, 5\$, 3\$)$ (valorile monedelor nu trebuie să fie sortate!), vom obține următoarele valori pentru elementele listelor $nrmin$ și $pred$ (am notat cu $+\infty$ valoarea $P + 1 = 13$):

i	0	1	2	3	4	5	6	7	8	9	10	11	12
nrmin	0	+∞	1	1	2	1	2	2	2	3	2	3	3
pred	-1	-1	2	3	2	5	3	2	5	2	5	5	2

Valorile evidențiate din listele $nrmin$ și $pred$ au fost calculate astfel:

- $nrmin[1] = +\infty$ și $pred[1] = -1$, deoarece niciuna dintre monedele cu valorile 2\$, 5\$ și 3\$ nu poate fi utilizată pentru a plăti suma $i = 1\$$;
- $nrmin[4] = 2$ și $pred[4] = 2$, deoarece doar monedele cu valorile 2\$ și 3\$ pot fi utilizate pentru a plăti suma $i = 4\$$ și $nrmin[4] = 1 + \min\{nrmin[4-2], nrmin[4-3]\} = 1 + \min\{nrmin[2], nrmin[1]\} = 1 + \min\{1, +\infty\} = 2$, deci minimul a fost obținut pentru moneda cu valoarea 2\$;
- $nrmin[7] = 2$ și $pred[7] = 2$, deoarece toate monedele pot fi utilizate pentru a plăti suma $i = 7\$$ și $nrmin[7] = 1 + \min\{nrmin[7-2], nrmin[7-5], nrmin[7-3]\} = 1 + \min\{nrmin[5], nrmin[2], nrmin[4]\} = 1 + \min\{1,1,2\} = 2$, deci minimul a fost obținut pentru moneda cu valoarea 2\$;
- $nrmin[12] = 3$ și $pred[12] = 2$, deoarece toate monedele pot fi utilizate pentru a plăti suma $i = 12\$$ și $nrmin[12] = 1 + \min\{nrmin[12-2], nrmin[12-5], nrmin[12-3]\} = 1 + \min\{nrmin[10], nrmin[7], nrmin[9]\} = 1 + \min\{2,2,3\} = 3$, deci minimul a fost obținut pentru moneda cu valoarea 2\$.

Numărul minim de monede necesare pentru a plăti suma $P = 12\$$ folosind monede cu valorile $v = (2\$, 5\$, 3\$)$ este $nrmin[12] = 3$, iar pentru a reconstitui o modalitate optimă de plată vom utiliza informațiile din lista $pred$, astfel:

- inițializăm un indice i cu P, deci $i = 12$ (variabila i reprezintă suma curentă de plată);
- $pred[i] = pred[12] = 2 \neq -1$, deci pentru a plăti suma $i = 12\$$ folosind un număr minim de monede a fost utilizată o monedă cu valoarea de 2\$, pe care o afișăm, și apoi indicele i devine egal cu $i - pred[i] = 10$ (suma de plată rămasă);
- $pred[i] = pred[10] = 5 \neq -1$, deci pentru a plăti suma $i = 10\$$ folosind un număr minim de monede a fost utilizată o monedă cu valoarea de 5\$, pe care o afișăm, și apoi indicele i devine egal cu $i - pred[i] = 5$ (suma de plată rămasă);
- $pred[i] = pred[5] = 5 \neq -1$, deci pentru a plăti suma $i = 5\$$ folosind un număr minim de monede a fost utilizată o monedă cu valoarea de 5\$, pe care o afișăm, și apoi indicele i devine egal cu $i - pred[i] = 0$ (suma de plată rămasă);
- $pred[i] = pred[0] = -1$, deci am terminat de afișat o modalitate de plată a sumei folosind un număr minim de monede și ne oprim.

În continuare, vom prezenta implementarea acestui algoritm în limbajul Python, considerând faptul că datele de intrare se citesc din fișierul text `monede.txt`, care conține pe prima linie valorile monedelor, iar pe a doua linie se află suma de plată P:

```
# citim datele de intrare din fișierul text "monede.txt"
# pe prima linie avem valorile monedelor (elementele listei v)
f = open("monede.txt")
aux = f.readline()
v = [int(x) for x in aux.split()]
# a doua linie conține suma de plată P
aux = f.readline()
P = int(aux)

# inițializăm listele nrmin și pred
nrmin = [P+1] * (P+1)
nrmin[0] = 0
pred = [-1] * (P+1)
# calculăm valorile nrmin[1],..., nrmin[P]
# folosind relația de recurență prezentată
for suma in range(1, P+1):
    for moneda in v:
        if moneda <= suma and 1 + nrmin[suma-moneda] < nrmin[suma]:
            nrmin[suma] = 1 + nrmin[suma-moneda]
            pred[suma] = moneda

# afișăm datele de ieșire
if nrmin[P] == P+1:
    print("Suma", P, "nu poate fi platita!")
else:
    print("Numărul minim de monede necesare pentru a plăti suma", P,
"este", nrmin[P])
    print("O modalitate de plată:")
    suma = P
    while pred[suma] != -1:
        print(pred[suma], end=" ")
        suma = suma - pred[suma]
```

Algoritmul prezentat utilizează varianta înapoi a tehnicii programării dinamice, iar complexitatea sa este egală cu $\mathcal{O}(nP)$. O astfel de complexitate se numește *complexitate pseudo-polinomială*, deoarece P nu reprezintă o dimensiune a datelor de intrare, ci o valoare a unei date de intrare! Pentru a exprima complexitatea acestui algoritm doar în raport de dimensiunile datelor de intrare vom folosi faptul ca un număr întreg strict pozitiv x poate fi reprezentat în formă binară folosind minim $1 + [\log_2 x]$ biți, deci complexitatea acestui algoritm este, de fapt, $\mathcal{O}\left(n2^{1+[\log_2 P]}\right) \approx \mathcal{O}\left(n2^{[\log_2 P]}\right)$, ceea ce înseamnă că are *o complexitate liniară în raport cu numărul n de monede* și *o complexitate exponențială în raport cu lungimea reprezentării binare a sumei P de plată*!

2. Problema rucsacului (varianta discretă)

Considerăm un rucsac având capacitatea maximă G și n obiecte $O_1, O_2, \dots, O_n$ pentru care cunoaștem greutățile lor $g_1, g_2, \dots, g_n$ și câștigurile $c_1, c_2, \dots, c_n$ obținute prin încărcarea lor în rucsac. Știind faptul că toate greutățile și toate câștigurile sunt numere naturale nenule, iar orice obiect poate fi încărcat doar complet în rucsac (nu poate fi "tăiat"), să se determine o modalitate de încărcare a rucsacului astfel încât câștigul total obținut să fie maxim.

De exemplu, considerând $G = 10$ kg și $n = 5$ obiecte O_1, O_2, O_3, O_4, O_5 având câștigurile $c = (80, 50, 400, 60, 70)$ RON și greutățile $g = (5, 2, 20, 3, 4)$ kg, putem obține un câștig maxim egal cu 190 RON, încărcând obiectele O_1, O_2 și O_4.

În capitolul dedicat tehnicii de programare Greedy am văzut faptul că varianta fracționară a acestei probleme poate fi rezolvată corect utilizând tehnica respectivă. În cazul variantei discrete, tehnica Greedy nu va mai furniza o soluție corectă întotdeauna. Astfel, câștigurile unitare ale obiectelor din exemplul de mai sus vor fi $u = (16, 25, 20, 20, 17.5)$ RON/kg. Astfel, algoritmul Greedy ar selecta obiectele O_2, O_4 și O_5, deoarece obiectele nu pot fi "tăiate" în varianta discretă a problemei rucsacului, și ar obține un câștig total egal cu 180 RON, evident mai mic decât cel maxim de 190 RON!

Se observă foarte ușor faptul că varianta discretă a problemei rucsacului nu are întotdeauna soluție, respectiv în cazul în care greutatea celui mai mic obiect este strict mai mare decât capacitatea G a rucsacului, în timp ce varianta fracționară ar avea soluție în acest caz (ar "tăia" din obiectul cu cel mai mare câștig unitar o bucată cu greutatea G).

Pentru a rezolva problema folosind metoda programării dinamice, vom proceda într-un mod asemănător cu cel utilizat pentru a rezolva problema plății unei sume folosind un număr minim de monede, astfel:

- considerăm faptul că am analizat, pe rând, obiectele $O_1, O_2, \dots, O_{n-1}$ și am calculat câștigul maxim pe care îl putem obține folosindu-le (nu neapărat pe toate!) în limita întregii capacități G a rucsacului, deci mai trebuie să calculăm doar câștigul maxim pe care îl putem obține folosind și ultimul obiect O_n;
- dacă obiectul O_n nu încape în rucsac (deci $g_n > G$), înseamnă că nu-l putem folosi deloc, deci câștigul maxim rămâne cel pe care l-am obținut deja utilizând obiectele $O_1, O_2, \dots, O_{n-1}$;
- dacă obiectul O_n încape în rucsac (deci $g_n \leq G$), înseamnă că trebuie să decidem dacă este rentabil să-l încărcăm sau nu, comparând câștigul maxim deja obținut folosind obiectele $O_1, O_2, \dots, O_{n-1}$ în limita întregii capacități G a rucsacului cu câștigul care s-ar obține prin încărcarea obiectului O_n, respectiv cu suma dintre c_n și câștigul maxim care se poate obține folosind obiectele $O_1, O_2, \dots, O_{n-1}$ în limita capacității $G - g_n$ rămase în rucsac. Deoarece $1 \leq g_n \leq G$, rezultă că trebuie să cunoaștem câștigurile maxime care se pot obține folosind obiectele $O_1, O_2, \dots, O_{n-1}$ în limita oricărei capacități cuprinse între 0 și G-1, la care se adaugă câștigul maxim care se poate obține folosind obiectele $O_1, O_2, \dots, O_{n-1}$ în limita întregii capacități G a rucsacului (pentru cazul anterior), deci, de fapt, trebuie să cunoaștem câștigurile maxime care se pot obține folosind obiectele $O_1, O_2, \dots, O_{n-1}$ în limita oricărei capacități cuprinse între 0 și G!
- pentru a calcula câștigurile maxime care se pot obține folosind primele $n - 1$ obiecte $O_1, O_2, \dots, O_{n-1}$ în limita oricărei capacități cuprinse între 0 și G vom repeta raționamentul anterior pentru obiectul O_{n-1} și obiectele $O_1, O_2, \dots, O_{n-2}$, apoi pentru obiectul O_{n-2} și obiectele $O_1, O_2, \dots, O_{n-3}$ și așa mai departe, până când vom

calcula câştigurile maxime care se pot obţine folosind doar primul obiect O_1 în limita oricărei capacităţi cuprinse între 0 şi G.

În concluzie, pentru a rezolva problema utilizând tehnica programării dinamice, trebuie să cunoaştem toate câştigurile maxime care se pot obţine folosind primele i obiecte ($i \in \{0,1,\dots,n\}$), în limita oricărei capacităţi j cuprinse între 0 şi G, deci, aplicând tehnica memoizării, vom considera un tablou bidimensional $cmax$ cu $n+1$ linii şi $G+1$ coloane în care un element $cmax[i][j]$ va memora câştigul maxim care se poate obţine folosind primele i obiecte în limita a j kilograme. Astfel, relaţia de recurenţă care caracterizează substructura optimală a problemei este următoarea:

$$cmax[i][j] = \begin{cases} 0, & \text{dacă i} = 0 \text{ sau } j = 0 \\ cmax[i-1][j], & \text{dacă } g_i > j \\ max\{cmax[i-1][j], c[i] + cmax[i-1][j - g[i]]\}, & \text{dacă } g_i \le j \end{cases}$$

pentru fiecare $i \in \{0,1,\dots,n\}$ şi fiecare $j \in \{0,1,\dots,G\}$. În plus faţă de modalitatea de calcul a elementului $cmax[i][j]$ descrisă mai sus, am adăugat cazurile particulare $cmax[0][j] = cmax[i][0] = 0$ (evident, câştigul maxim $cmax[0][j]$ care se poate obţine folosind 0 obiecte în limita oricărei capacităţi j este 0 şi câştigul maxim $cmax[i][0]$ care se poate obţine folosind primele i obiecte în limita unei capacităţi nule este tot 0). De asemenea, am considerat tablourile c şi g ca fiind indexate de la 1, pentru a păs

Considerând exemplul dat, vom obţine următoarele valori pentru elementele matricei $cmax$:

	c_i	g_i	i/j	0	1	2	3	4	5	6	7	8	9	10
			0	0	0	0	0	0	0	0	0	0	0	0
O_1	80	5	1	0	0	0	0	0	80	80	80	80	80	80
O_2	50	2	2	0	0	50	50	50	80	80	130	130	130	130
O_3	400	20	3	0	0	50	50	50	80	80	130	130	130	130
O_4	60	3	4	0	0	50	60	60	110	110	130	140	140	190
O_5	70	4	5	0	0	50	60	70	110	120	130	140	180	190

Elementele evidenţiate în matricea $cmax$ au fost calculate astfel:

- $cmax[1][1] = cmax[1][2] = cmax[1][3] = cmax[1][4] = 0$, deoarece obiectul O_1 are greutatea $g_1 = 5$, deci poate fi încărcat doar în cazul în care capacitatea j a rucsacului este cel puţin egală cu 5 (de exemplu, folosind relaţia de recurenţă, obţinem $cmax[1][2] = cmax[0][2] = 0$), caz în care am obţinut $cmax[1][5] = \dots = cmax[1][10] = 80$ (de exemplu, folosind relaţia de recurenţă, obţinem $cmax[1][9] = \max\{cmax[0][9], c[1] + cmax[0][9-5]\} = \max\{0, 80+0\} = 80$);

- $cmax[2][7] = \max\{cmax[1][7], c[2] + cmax[1][7-2]\} = \max\{80, 50 + 80\} = 130$, deoarece în limita a $j = 7$ kg încap ambele obiecte O_1 și O_2;
- linia 3 este egală cu linia 2, deoarece $g_3 = 20$ kg $> G = 10$ kg, deci obiectul O_3 nu se poate încărca în niciun caz în rucsac;
- $cmax[4][10] = \max\{cmax[3][10], c[4] + cmax[3][10-3]\} = \max\{130, 60 + 130\} = 190$, deoarece în limita a $j = 10$ kg se poate adăuga obiectul O_4 la obiectele O_1 și O_2 care au fost încărcate pentru a obține câștigul maxim folosind primele $i = 3$ obiecte în limita a $j = 7$ kg;
- $cmax[5][9] = \max\{cmax[4][9], c[5] + cmax[4][9-4]\} = \max\{140, 70 + 110\} = 180$, deoarece în limita a $j = 9$ kg este mai rentabil să încărcăm obiectul O_5 alături de obiectele O_2 și O_4 (pentru care s-a obținut câștigul maxim de 110 RON folosind primele $i = 4$ obiecte în limita a $j = 5$ kg) decât să nu-l încărcăm, caz în care am păstra câștigul maxim de 140 RON obținut prin încărcarea obiectelor O_1 și O_4 dintre primele $i = 4$ obiecte în limita a $j = 9$ kg.

Câștigul maxim care se poate obține folosind toate cele n obiecte este dat de valoarea elementului $cmax[n][G]$, iar pentru a reconstitui o modalitate optimă de încărcare a rucsacului vom utiliza informațiile din matricea $cmax$, astfel:

- considerăm doi indici $i = n$ și $j = G$;
- dacă $cmax[i][j] = cmax[i-1][j]$, înseamnă fie că obiectul O_i nu încape în rucsac, fie încape, dar nu ar fi fost rentabil să-l încărcăm. Indiferent de motiv, obiectul O_i nu a fost încărcat în rucsac (nu face parte din soluția optimă), deci trecem la următorul obiect O_{i-1}, decrementând valoarea indicelui i;
- dacă $cmax[i][j] \neq cmax[i-1][j]$, înseamnă că a fost rentabil să încărcăm obiectul O_i în limita a j kg, deci îl afișăm și trecem la reconstituirea soluției optime pentru restul de $j - g[i]$ kg folosind obiectele $O_1, O_2, \dots, O_{i-1}$, scăzând din indicele j valoarea $g[i]$ și decrementând indicele i.

Se observă faptul că obiectele se vor afișa în ordinea descrescătoare a indicilor lor (în "sens invers"), deci trebuie utilizată o structură de date auxiliară sau o funcție recursivă pentru a le afișa în ordinea crescătoare a indicilor lor!

În cazul exemplului de mai sus, avem $cmax[5][10] = 190$, deci profitul maxim care se poate obține este de 190 RON, iar pentru reconstituirea unei modalități optime de încărcare a rucsacului vom urma traseul marcat cu roșu în matricea $cmax$, obiectele care se vor încărca în rucsac corespunzând liniilor pe care se află elementele încadrate cu un dreptunghi, respectiv obiectele O_1, O_2 și O_4:

	c_i	g_i	i/j	0	1	2	3	4	5	6	7	8	9	10
			0	0	0	0	0	0	0	0	0	0	0	0
O_1	80	5	1	0	0	0	0	0	$\boxed{80}$	80	80	80	80	80
O_2	50	2	2	0	0	50	50	50	80	80	$\boxed{130}$	130	130	130
O_3	400	20	3	0	0	50	50	50	80	80	130	130	130	130
O_4	60	3	4	0	0	50	60	60	110	110	130	140	140	$\boxed{190}$
O_5	70	4	5	0	0	50	60	70	110	120	130	140	180	190

În continuare, vom prezenta implementarea acestui algoritm în limbajul Python, considerând faptul că datele de intrare se citesc din fișierul text `obiecte.txt`, care conține pe prima linie capacitatea G a rucsacului, iar pe fiecare dintre următoarele n linii se află greutatea și câștigul câte unui obiect. Datele de ieșire, respectiv o modalitate optimă de încărcare a rucsacului, se vor scrie în fișierul text `rucsac.txt`.

```
# fișierul de intrare conține pe prima linie
# capacitatea G a rucsacului
f = open("obiecte.txt")
G = int(f.readline())

# greutățile obiectelor se vor memora într-o listă g, iar
# câștigurile lor într-o listă c
# ambele liste trebuie să fie indexate de la 1, deci adăugăm
# în fiecare câte o valoarea "inexistentă" egală cu 0
g = [0]
c = [0]
# pe fiecare dintre liniile rămase, fișierul text conține
# greutatea și câștigul unui obiect
for linie in f:
    aux = linie.split()
    g.append(int(aux[0]))
    c.append(int(aux[1]))

f.close()

# n = numărul de obiecte
n = len(g) - 1

# inițializăm toate elementele matricei cmax cu 0
cmax = [[0 for x in range(G+1)] for x in range(n+1)]

# calculăm elementele matricei cmax folosind relația de recurență
for i in range(1, n+1):
for i in range(1, n+1):
    for j in range(1, G+1):
        cmax[i][j] = cmax[i-1][j]
        if g[i] <= j and c[i]+cmax[i-1][j-g[i]] > cmax[i-1][j]:
            cmax[i][j] = c[i]+cmax[i-1][j-g[i]]

# scriem în fișierul text rucsac.txt o modalitate optimă
# de încărcare a rucsacului
f = open("rucsac.txt", "w", encoding="UTF-8")

f.write("Câștigul maxim: " + str(cmax[n][G]) + "\n")
f.write("Obiectele încărcate: ")
i, j = n, G
while i != 0:
    if cmax[i][j] != cmax[i-1][j]:
```

```
        f.write(str(i) + " ")
        j = j - g[i]
    i = i - 1

f.close()
```

Algoritmul prezentat utilizează varianta înapoi a tehnicii programării dinamice, iar complexitatea sa este una de tip pseudo-polinomial, fiind egală cu $\mathcal{O}(nG) \approx \mathcal{O}\left(n2^{[\log_2 G]}\right)$.